

Project 1: Exploratory Data Analysis of Historical Stock Market Data

This report answers the questions in *Project 1: Exploratory Data Analysis of Historical Stock Market Data* using figures taken directly from notebook output.

All fourteen parts are answered from figures and charts produced in the notebook. The dataset is `historical_stock_prices.csv` (20,973,889 rows, 8 columns) joined to `historical_stocks.csv` (6,460 rows, 5 columns) on `ticker`, covering 2 January 1970 to 24 August 2018.

Part 1: The datasets

1. What does `historical_stocks` represent? A reference table of securities. One row per listed security, holding its ticker, the exchange it trades on, the company name, its sector and its industry. It describes *what* each security is, with no time dimension at all.

List top 5 of Historical Stocks

	ticker	exchange	name	sector	\
0	PIH	NASDAQ	1347 PROPERTY INSURANCE HOLDINGS, INC.	FINANCE	
1	PIHPP	NASDAQ	1347 PROPERTY INSURANCE HOLDINGS, INC.	FINANCE	
2	TURN	NASDAQ	180 DEGREE CAPITAL CORP.	FINANCE	
3	FLWS	NASDAQ	1-800 FLOWERS.COM, INC.	CONSUMER SERVICES	
4	FCCY	NASDAQ	1ST CONSTITUTION BANCORP (NJ)	FINANCE	

	industry
0	PROPERTY-CASUALTY INSURERS
1	PROPERTY-CASUALTY INSURERS
2	FINANCE/INVESTORS SERVICES
3	OTHER SPECIALTY STORES
4	SAVINGS INSTITUTIONS

2. What does `historical_stock_prices` represent? Daily trading activity. One row per security per trading day, holding the opening, closing, adjusted closing, low and high prices plus the volume traded. It describes *how each security behaved* on a given day.

List top 5 of Historical Stock Prices

	ticker	open	close	adj_close	low	high	volume	date
0	AHH	11.50	11.58	8.493155	11.25	11.68	4633900.0	2013-05-08
1	AHH	11.66	11.55	8.471151	11.50	11.66	275800.0	2013-05-09
2	AHH	11.55	11.60	8.507822	11.50	11.60	277100.0	2013-05-10
3	AHH	11.63	11.65	8.544494	11.55	11.65	147400.0	2013-05-13
4	AHH	11.60	11.53	8.456484	11.50	11.60	184100.0	2013-05-14

3. What is the purpose of the `ticker` column? It is the unique identifier for a security and the join key between the two files. In the price file it is not unique on its own, a ticker appears once per trading day, so the business key there is `ticker + date`.

4. How are the two datasets related? One-to-many. Each row in `historical_stocks` corresponds to many rows in `historical_stock_prices`. A left join on `ticker` attaches sector and exchange information to every price observation.

The merge was performed and produced 20,973,889 rows, identical to the price table, confirming no row inflation, which is expected since `historical_stocks` has no duplicate tickers.

List top 5 of Historical Stock Prices					List top 5 of Historical Stock			
	ticker	open	close	adj_close		ticker	exchange	
0	AHH	11.50	11.58	8.493155	11	0	PIH	NASDAQ 1347 PROPER
1	AHH	11.66	11.55	8.471151	11	1	PIHPP	NASDAQ 1347 PROPER
2	AHH	11.55	11.60	8.507822	11	2	TURN	NASDAQ
3	AHH	11.63	11.65	8.544494	11	3	FLWS	NASDAQ
4	AHH	11.60	11.53	8.456484	11	4	FCCY	NASDAQ 15

Part 2: Dataset structure

`historical_stock_prices`

Name Ayleen Santander

GitHub Link:

https://github.com/AyleenSantander/M.11_Intro_AI/blob/main/Project.1_Data_Collection_and_Initial_Analysis_of_Stock_Market.pdf

```
<class 'pandas.DataFrame'>
RangeIndex: 20973889 entries, 0 to 20973888
Data columns (total 8 columns):
#   Column      Dtype
---  -
0   ticker      str
1   open         float64
2   close        float64
3   adj_close    float64
4   low          float64
5   high         float64
6   volume       float64
7   date         str
dtypes: float64(6), str(2)
memory usage: 1.3 GB
```

Missing values in Historical Stock Prices:

```
ticker      0
open        0
close       0
adj_close   0
low         0
high        0
volume      0
date        0
```

dtype: int64

Statistical Values of Historical Stock Prices:

	count	mean	std	min	25%	\
open	20973889.0	7.605823e+01	2.849639e+03	4.000000e-04	7.50	
close	20973889.0	7.611403e+01	2.870159e+03	2.000000e-04	7.50	
adj_close	20973889.0	1.481184e+14	4.574674e+16	2.282650e-09	4.62	
low	20973889.0	7.422064e+01	2.746059e+03	1.000000e-04	7.36	
high	20973889.0	7.803857e+01	2.997937e+03	4.000000e-04	7.63	
volume	20973889.0	1.227043e+06	1.316686e+07	1.000000e+00	22100.00	

	50%	75%	max
open	15.450000	29.719999	2.034000e+06
close	15.450000	29.719999	1.779750e+06
adj_close	11.381994	24.720465	1.894962e+19
low	15.240000	29.280001	1.440000e+06
high	15.660000	30.100000	2.070000e+06
volume	126000.000000	607400.000000	4.483504e+09

historical_stocks

Property

Name Ayleen Santander

GitHub Link:

https://github.com/AyleenSantander/M.11_Intro_AI/blob/main/Project.1_Data_Collection_and_Initial_Analysis_of_Stock_Market.pdf

```
<class 'pandas.DataFrame'>
RangeIndex: 6460 entries, 0 to 6459
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  -
0   ticker      6460 non-null   str
1   exchange    6460 non-null   str
2   name        6460 non-null   str
3   sector      5020 non-null   str
4   industry    5020 non-null   str
dtypes: str(5)
memory usage: 252.5 KB
Missing values in Historical Stock Prices:
ticker      0
exchange    0
name        0
sector      1440
industry    1440
dtype: int64
Statistical Values of Historical Stock Prices:
count unique      top  freq
ticker      6460   6460                PIH      1
exchange    6460     2             NASDAQ  3308
name        6460  5462  BANK OF AMERICA CORPORATION  16
sector      5020    13              FINANCE  1022
industry    5020   136    MAJOR PHARMACEUTICALS  419
```

The size difference is the point: 6,460 securities generate almost 21 million daily observations across roughly 48 years.

Part 3: Missing values

1. Which columns contain missing values? None in the price table. In the securities table, sector is missing for 1,440 records.

2. How many? historical_stock_prices returned zero nulls across all eight columns. historical_stocks has 1,440 missing sectors out of 6,460 securities, 22.3%.

```
#Missing values in the STOCKS dataset (you only checked prices)
print(hist_stock.isnull().sum())
```

```
ticker      0
exchange    0
name        0
sector      1440
industry    1440
dtype: int64
```

When the two tables are joined, 2,549,449 price rows (12.2% of all observations) end up with no sector label. The proportion is lower than 22.3% because securities lacking sector data tend to have shorter price histories.

The first tickers without a sector match are USAU, ASA, JHI, ADX, MCI, BIF, INSI, JHS, TY, GAM, WRE, MXF, VBF, KF, ASG, EEA, GAB, ZF, USA and MFM. Several of these are closed-end funds and trusts rather than operating companies.

3. Are missing values necessarily errors? No. A closed-end fund, ETF or trust has no operating sector, so a blank field is the correct representation, not a defect. The absence is informative in itself.

4. Should every missing value automatically be replaced? No. Imputing a sector for a fund would invent a fact the source never asserted, and any later sector-level aggregation would inherit that invention. The right treatment here is to retain the records and either exclude them from sector breakdowns or group them under an explicit "Unclassified" label. Deletion would discard 2.5 million valid price observations to fix a field that is not needed for price analysis at all.

Part 4: Duplicate records

1. Are there completely duplicated rows? No. `stock_prices.duplicated().sum()` returned 0.

2. Are there duplicate ticker + date combinations? No.

`duplicated(subset=['ticker', 'date']).sum()` also returned 0. In the securities table, `hist_stock['ticker'].duplicated().sum()` returned 0, so every security appears once.

3. Why is checking ticker + date different from checking the whole row? A full-row check only catches records identical in *every* field. The business key check catches a different and more serious problem: two records claiming to describe the same security on the same day but disagreeing on price or volume.

A full-row duplicate is a storage issue. A business-key duplicate is a correctness issue. Both returned clean here, which means the source file is well formed at the record level. The quality problems in this dataset sit inside the values, not in the row structure.

Part 5: Obvious data-quality problems

1. What data-quality issue was identified? Two checks were run and both came back **clean**:

- `hist_stock[hist_stock['ticker'].str.contains('ticker', case=False)]` returned an empty DataFrame, no accidental header row in the securities table.
- `stock_prices[stock_prices['ticker'].str.len() > 6]['ticker'].unique()` returned an empty array, every ticker is six characters or fewer, so no malformed or concatenated symbols.

No records were removed on these grounds. That is the correct outcome to report: the check was performed, found nothing, and nothing was deleted.

The real data-quality issue surfaced elsewhere. During loading, pandas raised `DtypeWarning: Columns (3) have mixed types on column index 3, which is adj_close`. A numeric column reporting mixed types means some rows held values pandas could not parse as numbers on a first pass. That warning is the earliest sign of the adjustment-factor corruption documented in Part 10.

2. Why should an obvious accidental header row be removed? Because it is not an observation. A row where `ticker` reads "ticker" and `close` reads "close" describes the file's layout, not a security's trading day. Leaving it in forces the numeric columns to object dtype, breaks every arithmetic operation, and inflates the row count by one record that corresponds to nothing in the real world.

3. Why should a legitimate security with missing sector information generally be retained? Because the missing field is metadata, not the observation itself. A closed-end fund with no sector still has valid, verifiable daily prices and volumes. Dropping it would remove real trading data to tidy up a descriptive label.

4. What is the difference between an obvious data-entry error and a missing value? An error is a value that is present but wrong, it asserts something false, such as an adjusted close of 1.89×10^{19} dollars. A missing value asserts nothing; it simply records that the information was never supplied. Errors must be corrected or excluded because they corrupt calculations. Missing values can often be retained and worked around, because a null is honest about what it does not know.

Part 6: The date column

The `date` column loaded as a string and was converted with `pd.to_datetime(..., errors='coerce')`, then set as the DataFrame index and sorted. `stock_prices.dtypes` confirms `datetime64` on the index; no values were coerced to `NaT`.

1. Earliest date: 2 January 1970.

2. Latest date: 24 August 2018.

```

: #Fix te date column and adj_close
stock_prices['date'] = pd.to_datetime(stock_prices['date'], errors='coerce')
stock_prices = stock_prices.sort_values(['ticker', 'date']).reset_index(drop=True)
print(stock_prices.sort_values(['ticker', 'date']))

print('--'*45)

stock_prices['adj_close'] = pd.to_numeric(stock_prices['adj_close'], errors='coerce')
print(stock_prices['adj_close'].isna().sum())

```

	ticker	open	close	adj_close	low	high \
0	A	32.546494	31.473534	27.494957	28.612303	35.765381
1	A	30.713520	28.880543	25.229753	28.478184	30.758226
2	A	29.551144	31.473534	27.494957	28.657009	31.473534
3	A	30.400572	28.612303	24.995413	28.612303	31.205294
4	A	28.701717	29.372318	25.659359	28.612303	29.998211
...	...	...	...	...	...	...
20973884	ZYNE	6.280000	6.130000	6.130000	6.010000	6.339000
20973885	ZYNE	6.130000	6.310000	6.310000	6.070000	6.500000
20973886	ZYNE	6.310000	6.260000	6.260000	6.235000	6.500000
20973887	ZYNE	6.400000	6.300000	6.300000	6.160000	6.460000
20973888	ZYNE	6.300000	6.300000	6.300000	6.260000	6.420000

	volume	date
0	62546300.0	1999-11-18
1	15234100.0	1999-11-19
2	6577800.0	1999-11-22
3	5975600.0	1999-11-23
4	4843200.0	1999-11-24
...	...	...
20973884	309100.0	2018-08-20
20973885	589300.0	2018-08-21
20973886	263200.0	2018-08-22
20973887	325900.0	2018-08-23
20973888	179200.0	2018-08-24

[20973889 rows x 8 columns]

0

That is 48 years and 8 months. Note this differs from the dataset's stated title of "1970 – 2020", the actual coverage ends in August 2018, which is why the assignment asks for the observed range rather than the advertised one.

3. Why convert from text to datetime? As text, dates sort alphabetically, so "10/01/1985" precedes "2/01/1970". As datetime, they sort chronologically, and pandas gains access to the year, month, quarter and weekday components.

4. What becomes easier? Resampling to monthly or annual frequency, slicing by date range, extracting the decade (`index.year // 10 * 10`), computing rolling windows and lagged returns, and plotting on a properly scaled time axis. The entire decade segmentation in this project depends on the conversion having been done first.

```
#1. Create a new column in the DataFrame for prices to represent the decade.
stock_prices['decade'] = (stock_prices.index.year // 10) * 10

print(stock_prices['decade'].value_counts().sort_index())
print('--'*45)
print(stock_prices.head())
```

```
decade
1970    164499
1980    1470571
1990    3427832
2000    6572053
2010    9338934
Name: count, dtype: int64
```

```
-----
              ticker      open      close  adj_close      low      high \
date
1970-01-02    CNP    11.099500    11.169750    0.107916    10.994125    11.204875
1970-01-02    CAT     3.541667     3.458333    0.378521     3.447917     3.541667
1970-01-02    KO     0.856771     0.856771    0.017364     0.850260     0.856771
1970-01-02    NAV   251.250000   261.250000   169.416168   251.250000   261.250000
1970-01-02    MMM     6.851562     6.851562    0.438697     6.843750     6.890625
```

```
              volume  decade
date
1970-01-02    24400.0    1970
1970-01-02   276000.0    1970
1970-01-02  1104000.0    1970
1970-01-02     900.0    1970
1970-01-02    72000.0    1970
```

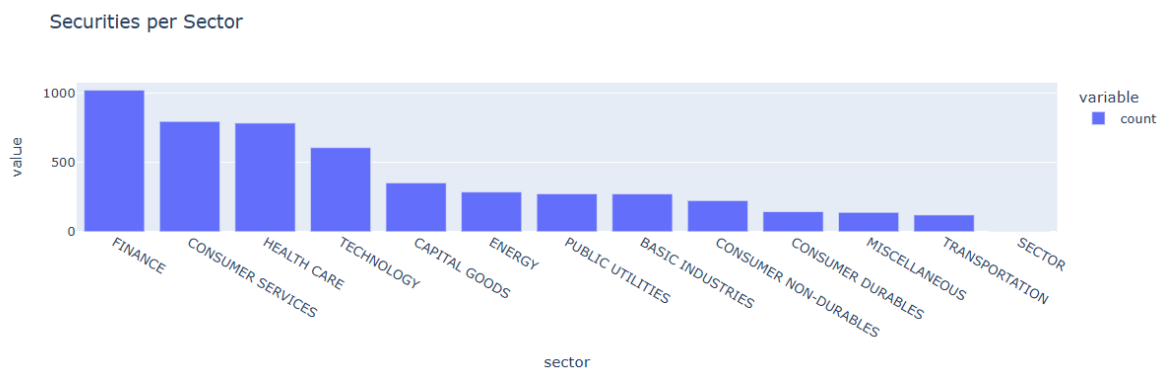
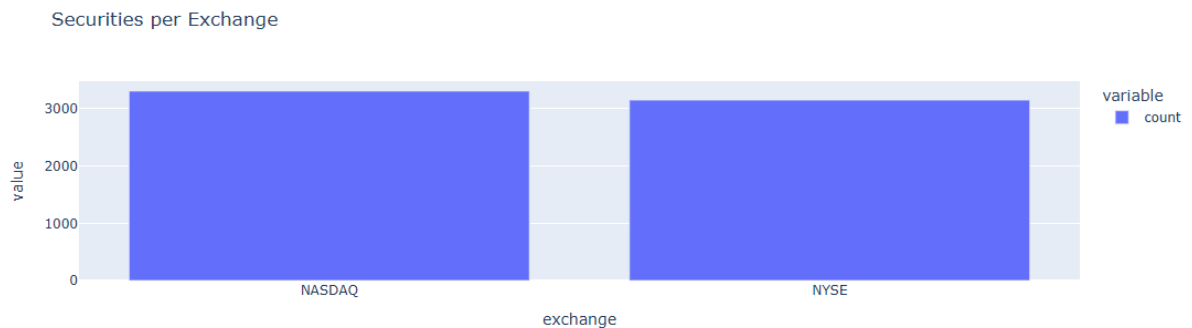
Part 7: Categorical variables

7.1 Stock exchanges

1. Which exchanges are represented? Two: NASDAQ and NYSE. No other venue appears, and no records are missing an exchange.

2. How many securities on each? The bar chart shows the two at roughly 3,300 and 3,150 against a total of 6,460, close to an even split, with NASDAQ marginally ahead.

3. Which has more? NASDAQ, but only just. The near-parity is worth a remark: NASDAQ typically lists far more small-cap issues than NYSE, so a balanced count suggests this dataset is a curated selection rather than a complete listing of either exchange. That is consistent with the survivorship bias noted later.



7.2 Sectors

1. Which sectors contain the most securities? From the bar chart, in descending order: **Finance** (roughly 1,000, clearly the largest), **Consumer Services** (~790), **Health Care** (~780), **Technology** (~600), **Capital Goods** (~350), **Energy** (~280), **Public Utilities** (~260), **Basic Industries** (~250), **Consumer Non-Durables** (~220), **Consumer Durables** (~140), **Miscellaneous** (~140), **Transportation** (~110).

Twelve sectors cover 5,020 securities, the 6,460 total minus the 1,440 with no sector. Finance alone accounts for about a fifth of all classified securities, which reflects how many banks, insurers, REITs and investment vehicles are listed.

2. How many securities have missing sector information? 1,440 of 6,460, 22.3%.

3. What could explain the missing sectors?

- **Non-operating entities.** Closed-end funds, ETFs, unit trusts and holding vehicles have no industrial sector by definition. The unmatched tickers seen in the merge, ASA, ADX, GAM, MXF, KF, GAB, are largely funds of this kind.
- **Delisted or acquired companies.** A security still present in the price history may have been dropped from the current reference file when it stopped trading, leaving a partial record.
- **Multiple share classes.** Preferred shares and warrants (such as PIHPP alongside PIH) sometimes carry the ticker but not the full classification.
- **Source coverage gaps.** The sector taxonomy may simply not have been populated for every listing at the time the file was compiled.

Part 8: Numerical variables

From `stock_prices.describe().T`, all across 20,973,889 observations:

Statistical Values of Historical Stock Prices:

	count	mean	std	min	25%	\
open	20973889.0	7.605823e+01	2.849639e+03	4.000000e-04	7.50	
close	20973889.0	7.611403e+01	2.870159e+03	2.000000e-04	7.50	
adj_close	20973889.0	1.481184e+14	4.574674e+16	2.282650e-09	4.62	
low	20973889.0	7.422064e+01	2.746059e+03	1.000000e-04	7.36	
high	20973889.0	7.803857e+01	2.997937e+03	4.000000e-04	7.63	
volume	20973889.0	1.227043e+06	1.316686e+07	1.000000e+00	22100.00	
	50%	75%	max			
open	15.450000	29.719999	2.034000e+06			
close	15.450000	29.719999	1.779750e+06			
adj_close	11.381994	24.720465	1.894962e+19			
low	15.240000	29.280001	1.440000e+06			
high	15.660000	30.100000	2.070000e+06			
volume	126000.000000	607400.000000	4.483504e+09			

1. Median closing price: 15.45

2. Mean closing price: 76.11

3. How different? The mean is 4.9 times the median, a gap of 60.66. Half of all observations fall below 15.45, yet the average sits at 76.11.

4. Largest standard deviation: `adj_close`, at 4.57×10^{16} . Among the ordinary price columns it is high at 2,997.94; `volume` has the largest raw spread of the non-price variables at 13.17 million.

5. Particularly large maxima: `adj_close` reaches 1.89×10^{19} , which is not a possible share price. `Open`, `high`, `low` and `close` all exceed one million dollars. `Volume` peaks at 4.48 billion shares in one day.

6. What does the mean-median gap tell us? The distributions are heavily right-skewed. The 75th percentile of `close` is 29.72 while the maximum is 1,779,750, the top quarter of the range stretches across six orders of magnitude. A mean pulled five times above the median means a small number of extreme observations dominate the arithmetic, so the median is the more representative measure of a typical security throughout this analysis.

The `adj_close` standard deviation exceeding its own mean by two orders of magnitude is not skew at all. It is a signal that the column contains impossible values, which Part 10 investigates.

Part 9: Investigating the highest closing prices

`stock_prices.nlargest(15, 'close')` returned:

```
#Highest CLOSE (you only did adj_close)
print(stock_prices.nlargest(15, 'close').reset_index()[
    ['ticker', 'date', 'open', 'high', 'low', 'close', 'volume']])
```

	ticker	date	open	high	low	close	volume
0	SCON	2000-02-28	1395000.000	1797750.000	1368000.00	1779750.0	400.0
1	SCON	2000-02-29	2034000.000	2070000.000	1440000.00	1595250.0	400.0
2	SCON	2000-03-13	900000.000	1404000.000	861750.00	1363500.0	300.0
3	CODA	2001-02-05	1354500.000	1736700.000	1354500.00	1354500.0	34.0
4	TVIX	2013-02-26	1375000.000	1542500.000	1300000.00	1347500.0	100.0
5	SCON	2000-03-02	1120500.000	1368000.000	1098280.75	1261125.0	200.0
6	SCON	2000-02-25	873561.625	1369125.000	873000.00	1215000.0	200.0
7	TOPS	2016-08-01	783000.000	1276200.000	783000.00	1189800.0	100.0
8	SCON	2000-03-01	1368000.000	1512000.000	1107000.00	1134000.0	300.0
9	SCON	2000-03-14	1352250.000	1354500.000	1080000.00	1111500.0	100.0
10	TVIX	2013-03-19	997500.000	1110000.000	985000.00	1007500.0	100.0
11	SCON	2000-03-07	900000.000	1065936.625	859500.00	997875.0	100.0
12	TVIX	2013-06-24	942500.000	975000.000	895000.00	950000.0	100.0
13	TVIX	2013-06-20	810000.000	962500.000	800000.00	915000.0	100.0
14	SCON	2000-03-06	1222875.000	1222875.000	885375.00	895500.0	200.0

1. Which securities? Four tickers dominate: **SCON** (Superconductor Technologies), **CODA**, **TVIX** (a volatility ETN) and **TOPS** (Top Ships).

2. On what dates? SCON clusters in late February and March 2000, the peak of the dot-com bubble. CODA in February 2001. TVIX across 2013. TOPS in August 2016.

3. What was the volume? This is the decisive detail. Volumes are 34, 100, 200, 300 and 400 shares. Against a dataset median of 126,000 shares per day, these are three to four orders of magnitude below normal.

4. Do these appear unusual? Yes, on two counts. No US-listed share has traded near 1.7 million dollars, Berkshire Hathaway Class A, the highest-priced US share in history, was under 330,000 dollars in 2018. And the internal consistency holds within each row: $low < close < high$ in every case, so these are not random corruptions but coherently scaled figures.

5. Can we conclude they are errors from this alone? No, and the coherence is exactly why. Two innocent explanations remain open:

- **Reverse-split back-adjustment.** TVIX and TOPS both executed multiple large reverse splits. A 1-for-10 reverse split applied repeatedly to a historical price series produces exactly this pattern: enormous back-adjusted prices attached to tiny back-adjusted volumes. Under this reading the numbers are *correct by construction* and simply mean something different from a nominal traded price.
- **A genuine thin-market print.** A security trading 34 shares in a day can print an extreme price without it being an error in the data.

Deciding requires outside evidence: the corporate-action history for each ticker, or the same dates from a second data source. The observations were therefore **retained** and flagged rather than deleted.

The volume pattern is itself the strongest evidence for the reverse-split explanation. A real price of 1.7 million dollars per share would attract enormous trading interest, not 400 shares.

Part 10: Investigating adj_close

This is the most significant data-quality finding in the project.

Detect

`describe()` reported an `adj_close` mean of 1.48×10^{14} against a median of 11.38, with a maximum of 1.89×10^{19} . No security has ever been worth 19 quintillion dollars.

Investigate

`nlargest(20, 'adj_close')` returned twenty consecutive AAN records from February and April 1987:

```
print(stock_prices.nlargest(20, 'adj_close')[['ticker', 'adj_close', 'close', 'volume']])
```

	date	ticker	adj_close	close	volume
	1987-02-27	AAN	1.894962e+19	1.296296	346900.0
	1987-03-10	AAN	1.894962e+19	1.296296	20200.0
	1987-03-11	AAN	1.894962e+19	1.296296	18900.0
	1987-03-12	AAN	1.894962e+19	1.296296	27000.0
	1987-03-17	AAN	1.894962e+19	1.296296	31000.0
	1987-03-18	AAN	1.894962e+19	1.296296	272700.0
	1987-03-19	AAN	1.894962e+19	1.296296	4000.0
	1987-03-25	AAN	1.894962e+19	1.296296	184900.0
	1987-03-27	AAN	1.881428e+19	1.287037	205200.0
	1987-03-26	AAN	1.854356e+19	1.268519	20200.0
	1987-03-04	AAN	1.840821e+19	1.259259	24300.0
	1987-03-09	AAN	1.840821e+19	1.259259	62100.0
	1987-03-20	AAN	1.840821e+19	1.259259	9400.0
	1987-03-23	AAN	1.840821e+19	1.259259	9400.0
	1987-04-01	AAN	1.840821e+19	1.259259	18900.0
	1987-03-02	AAN	1.813750e+19	1.240741	221400.0
	1987-03-13	AAN	1.813750e+19	1.240741	1300.0
	1987-03-16	AAN	1.813750e+19	1.240741	1300.0
	1987-03-30	AAN	1.813750e+19	1.240741	14800.0
	1987-04-06	AAN	1.813750e+19	1.240741	10800.0

2. How do close and adj_close compare? They diverge by nineteen orders of magnitude. `close` reads 1.296296, a perfectly ordinary penny-stock price, internally consistent with that row's open, high and low. Only the adjusted figure is impossible. This localizes the fault precisely: **the raw price data is sound; the adjustment factor is corrupt.**

Counting every row outside a plausible range confirms the fault is concentrated, not random:

Name Ayleen Santander

GitHub Link:

https://github.com/AyleenSantander/M.11_Intro_AI/blob/main/Project.1_Data_Collection_and_Initial_Analysis_of_Stock_Market.pdf

```
#Light version
mask = ~stock_prices['adj_close'].between(0, 100_000)
print(stock_prices.loc[mask, 'decade'].value_counts().sort_index())
print(stock_prices.loc[mask, 'ticker'].value_counts().head(10))
```

```
decade
1980    461
1990    574
2000   2621
2010    403
Name: count, dtype: int64
ticker
YRCW    1152
ELJ      1100
AAN       824
MBVX      207
SAB       194
TVIX      182
CODA      155
CTIC      100
ABIO       63
SCON       44
Name: count, dtype: int64
```

Three tickers, YRCW, ELJ and AAN , account for 3,076 of 4,059 affected rows, or **76%**. By decade: 1980s 461, 1990s 574, 2000s 2,621, 2010s 403. The 1970s are untouched.

Two distinct failure modes, not one

Inspecting `stock_prices[stock_prices['adj_close'] > 10_000]` shows the flagged rows split into two clearly different patterns:

Date	Ticker	close	adj_close	Agreement
1981-06-08	YRCW	72,187.50	71,858.12	agree within 0.5%
1983-07-11	YRCW	139,687.50	139,050.10	agree
1986-04-21	YRCW	271,875.00	270,636.20	agree
1987-01-20	AAN	1.055556	1.543041×10^{19}	diverge by 10^{19}

1987-02-04

AAN 1.111111 $\times 10^{19}$ 1.624253 $\times 10^{19}$ diverge by 10^{19}

```
bad = stock_prices[stock_prices['adj_close'] > 10]
print(bad[['ticker', 'close', 'adj_close']].head(
```

date	ticker	close	adj_close
1981-06-08	YRCW	72187.500000	7.185812e+04
1982-09-21	YRCW	74531.250000	7.419118e+04
1982-09-23	YRCW	74062.500000	7.372456e+04
1982-11-02	YRCW	72656.250000	7.232475e+04
1983-02-11	YRCW	87187.500000	8.678966e+04
1983-07-11	YRCW	139687.500000	1.390501e+05
1986-01-29	YRCW	245625.000000	2.445058e+05
1986-04-21	YRCW	271875.000000	2.706362e+05
1987-01-20	AAN	1.055556	1.543041e+19
1987-01-21	AAN	1.055556	1.543041e+19
1987-01-22	AAN	1.055556	1.543041e+19
1987-01-23	AAN	1.055556	1.543041e+19
1987-01-26	AAN	1.055556	1.543041e+19
1987-01-27	AAN	1.074074	1.570112e+19
1987-01-28	AAN	1.074074	1.570112e+19
1987-01-29	AAN	1.092593	1.597182e+19
1987-01-30	AAN	1.092593	1.597182e+19
1987-02-02	AAN	1.074074	1.570112e+19
1987-02-03	AAN	1.074074	1.570112e+19
1987-02-04	AAN	1.111111	1.624253e+19

YRCW is not a data error. Its `close` and `adj_close` agree closely; both are simply very large. YRC Worldwide executed extreme reverse splits, so its entire pre-split history is back-adjusted to enormous per-share figures. These values are *correct by construction* and mean "what one of today's shares corresponds to back then", not "what a share traded at". They were caught by the threshold filter only because the filter tests magnitude, not consistency.

AAN is a genuine error. `close` reads a perfectly ordinary 1.06 while `adj_close` reads 1.5×10^{19} on the same row. No corporate action produces that. This is a broken adjustment factor.

3. Do the values appear unusual? Beyond unusual, they are impossible. An adjusted close cannot exceed global GDP.

4. Over what period? Each affected ticker shows a contiguous block rather than scattered points. AAN runs across 1987; the pattern repeats per security. This is the signature of a single

broken corporate-action record propagating backwards through that ticker's entire adjusted history.

5. Should they automatically be deleted? No, and the process matters more than the outcome.

Detect → Investigate → Document → Decide was followed. Deleting on detection would have concealed the pattern that makes the diagnosis possible: that the fault is confined to ten securities, that `close` remains valid on the same rows, and that whole date ranges are affected together. Those facts are what identify this as an adjustment-factor failure rather than random corruption.

Decision taken: 4,059 rows (0.019% of the dataset) were excluded from `adj_close` analysis via `stock_prices['adj_close'].between(0, 100_000)`. The `close` values on those same rows remain valid and were kept for all price analysis. A threshold filter is a judgment call and is documented as such.

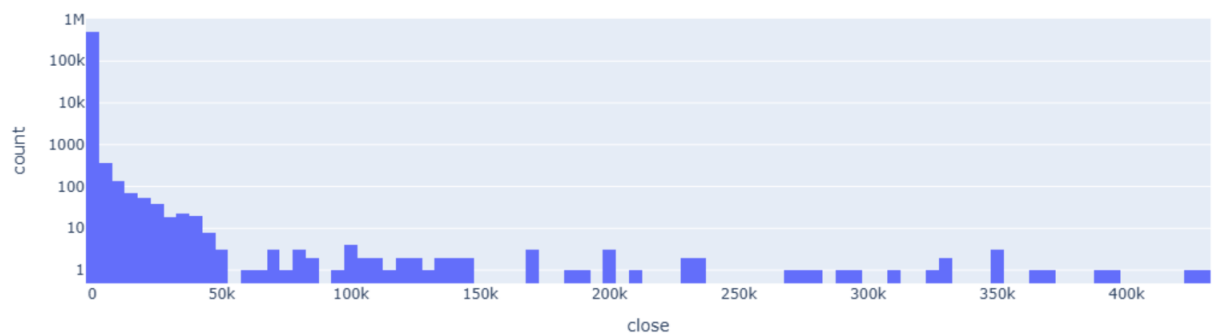
Parts 11 & 12: Distributions and boxplots

Both were produced on a 500,000-row random sample (`random_state=42`), since 21 million points will not render in a browser.

Part 11: Histograms

- 1. What does the closing-price distribution look like?** Exactly as the statistics predicted: a single towering bar against the left axis and then nothing visible. The chart uses a logarithmic *count* axis, and even so the first bin holds over 100,000 observations while bins past 50,000 dollars hold 1 to 10 each. Isolated bars scatter out to beyond 400,000 dollars.

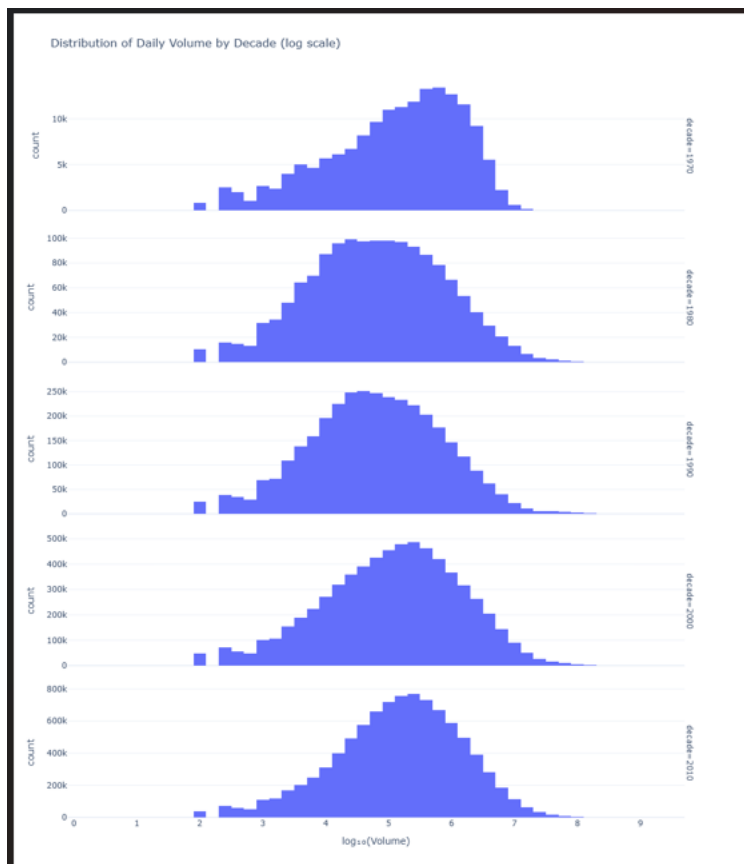
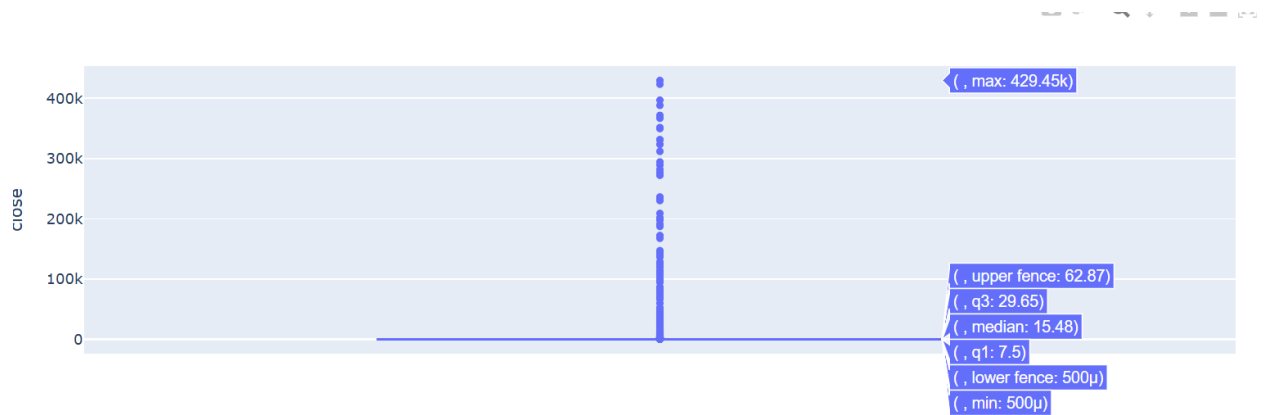
```
#Histograms and boxplots of close
samp = stock_prices.sample(500_000, random_state=42)
px.histogram(samp, x='close', nbins=100, log_y=True).show()
px.box(samp, y='close').show()
```

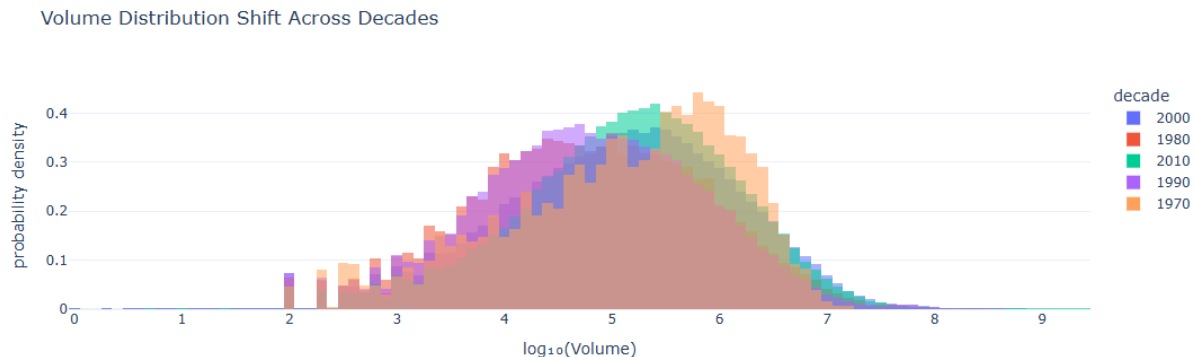


Name Ayleen Santander

GitHub Link:

https://github.com/AyleenSantander/M.11_Intro_AI/blob/main/Project.1_Data_Collection_and_Initial_Analysis_of_Stock_Market.pdf





2. Symmetric or skewed? Extremely right-skewed. Mean 76.11 against median 15.45 is decisive, for a symmetric distribution the two would coincide.

3. Are there extreme values? Yes, and the log count axis is what makes them visible. On a linear count axis those single-observation bars at 100,000+ would be invisible, and the distribution would look like one bar and empty space.

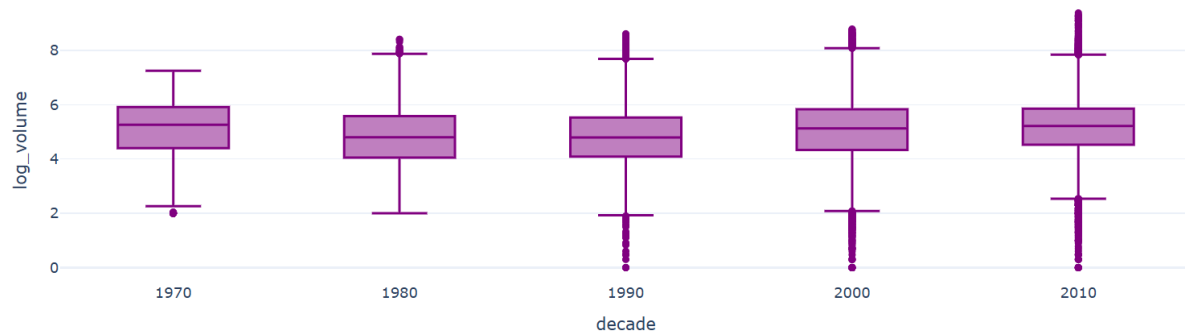
4. What does the volume distribution look like? Quite different, and this is the more interesting chart. Plotted as $\log_{10}(\text{volume})$ it is **roughly bell-shaped and near-symmetric** in every decade, centered around 4.7 to 5.3, that is, 50,000 to 200,000 shares. Trading volume is approximately **log-normal**: skewed in raw units, well behaved once logged.

The faceted-by-decade version shows the mode shifting: about 5.3 in the 1970s, dropping to roughly 4.7 in the 1980s and 1990s, then recovering to 5.2 and 5.3 in the 2000s and 2010s. The dip matches the arrival of thousands of thinly traded small-caps; the recovery matches liquidity broadening again. The overlaid density version makes the 1970s and 2010s curves sit noticeably to the right of the 1980s and 1990s.

The 2010s panel also shows a right tail reaching $\log_{10} = 9$, a single billion-share trading day.

5. How do the visuals support `.describe()`? They make the mean-median gap visible as shape rather than arithmetic. The numbers say skewed; the histogram shows *why* the mean is not a useful summary here.

Part 12: Boxplots



1. What does the box represent? The interquartile range , 25th to 75th percentile, containing the middle 50% of observations. For `close` that is 7.50 to 29.72. The line inside is the median.

2. What do the points outside the whiskers represent? Observations beyond $1.5 \times \text{IQR}$ from the quartiles. The IQR for `close` is 22.22, so the upper whisker reaches about 63. Every observation above that plots as an individual point.

3. What does the boxplot say about outliers? The `close` boxplot is the clearest possible illustration. The box itself is invisible, compressed to a flat line at the bottom of a scale running to 430,000, while a dense vertical column of individual points rises above it. Millions of observations sit beyond the upper whisker. This is a property of financial data, not a defect: the $1.5 \times \text{IQR}$ rule assumes an approximately normal distribution, which prices do not follow.

4. How does it compare with the histogram? They fail in the same direction and for the same reason. Both are unreadable on a linear scale here. The log-volume boxplot by decade, by contrast, works well, boxes are visible, medians are comparable, and the pattern is legible at a glance:

Decade Median $\log_{10}(\text{volume})$ Approx. shares Box height

1970s	~5.1	~125,000	narrowest
1980s	~4.75	~56,000	wider
1990s	~4.7	~50,000	wider
2000s	~5.1	~125,000	widest
2010s	~5.15	~140,000	widest

The medians dip in the 1980s and 1990s and recover afterwards, while the boxes widen steadily , the same widening-dispersion story the summary statistics told, now visible directly. The 1970s box is visibly the tightest, consistent with a sample of only 93 large, similar firms.

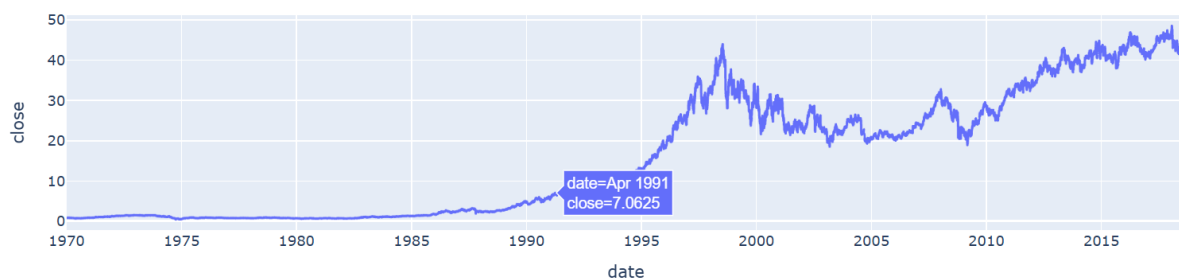
5. Why is a boxplot useful for financial data? It surfaces candidate observations for investigation quickly, and it compares groups side by side, the decade boxplot above conveys

both the median shift and the widening spread in one view, which no table does as economically. Two caveats: an outlier in finance is usually a real event rather than an error, and a boxplot of raw prices is useless without a log transform.

Part 13: One stock over time

1. Which ticker? KO. It appears in the very first rows of the dataset on 2 January 1970 at a close of 0.856771, survives the full 48-year span, and is a stable large-cap with no reverse splits, so the series is clean. SCON, TVIX, TOPS, YRCW, ELJ and AAN were avoided because they carry the anomalies documented in Parts 9 and 10.

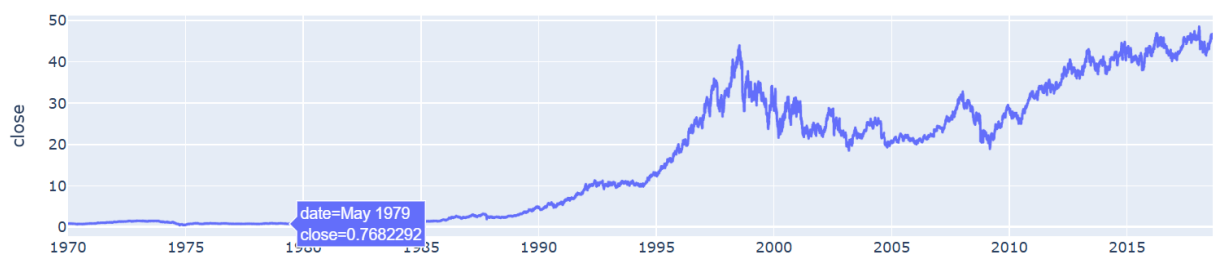
KO closing price, 1970–2018



2. How does the price change over time? The chart shows four clear regimes:

- **1970–1985, flat and low.** The price sits under 2 dollars for fifteen years with almost no visible movement. These are split-adjusted figures, so this reflects the cumulative effect of Coca-Cola's later splits rather than a genuinely stagnant company.

KO closing price, 1970–2018



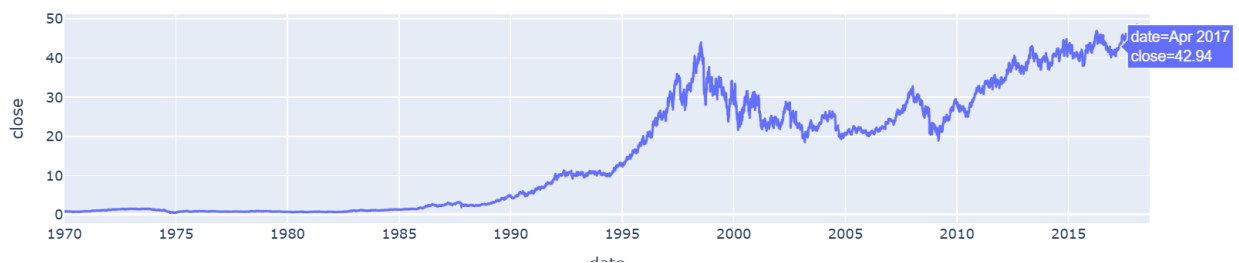
- **1985–1998, sustained growth.** A slow lift from about 2 through the late 1980s, then a steep climb from 1990, reaching roughly 43 dollars in mid-1998. This is the largest sustained run in the series.
- **1998–2003, a long decline.** From about 43 down to roughly 20, a fall of more than half, and notably deeper and longer than anything in the preceding thirty years.

- **2003–2018, recovery and steady growth.** Back up through the 2000s, a visible dip around 2009, then a steady climb to about 46 by August 2018, marginally above the 1998 peak.

3. Are there periods of unusually high or low prices? The 1998 peak at roughly 43 and the 2003 trough near 20 are the two standouts. Worth noting what is *absent*: the 1973–74 bear market and the 1987 crash are invisible at this scale, because the split-adjusted price was under 2 dollars then and a 30% move is imperceptible on a linear axis spanning 0 to 50. The 2008–09 decline is visible but modest, a dip from roughly 30 to 20, consistent with a defensive consumer-staples stock holding up better than the market.

4. Are there gaps in the data? The line is continuous from 1970 to 2018 with no visible breaks, and no trading halt or suspension is apparent. The expected weekday-only spacing does not show at this resolution. Confirm numerically with `ko.index.to_series().diff().dt.days` and look for values above 5.

KO closing price, 1970–2018



5. What additional information would explain major price changes?

- **Corporate actions** = split and dividend history, which is what would let you verify `adj_close` against `close`.
- **Earnings announcements** and guidance revisions.
- **Macroeconomic context** = interest rates, recession dates, inflation.
- **Sector and index movements**, to separate company-specific moves from market-wide ones.
- **News events** = mergers, litigation, regulatory decisions, management changes.

Name Ayleen Santander

GitHub Link:

https://github.com/AyleenSantander/M.11_Intro_AI/blob/main/Project.1_Data_Collection_and_Initial_Analysis_of_Stock_Market.pdf

```
comparison = clean_prices.groupby('decade').agg(
    n_rows=('adj_close', 'size'),
    n_tickers=('ticker', 'nunique'),
    close_mean=('adj_close', 'mean'),
    close_median=('adj_close', 'median'),
    close_std=('adj_close', 'std'),
    vol_mean=('volume', 'mean'),
    vol_median=('volume', 'median'),
    vol_std=('volume', 'std')
).round(2)

comparison['close_cv'] = (comparison['close_std'] / comparison['close_mean']).round(2)
comparison['vol_skew_ratio'] = (comparison['vol_mean'] / comparison['vol_median']).round(2)

print(comparison)
```

	n_rows	n_tickers	close_mean	close_median	close_std	vol_mean \
decade						
1970	164499	93	7.31	0.35	39.93	724893.64
1980	1470110	900	9.22	1.18	183.06	837750.58
1990	3427258	2117	30.29	4.87	779.01	863665.52
2000	6569431	3429	58.05	11.65	1057.91	1384100.65
2010	9338531	5685	52.54	17.79	699.74	1320560.15

	vol_median	vol_std	close_cv	vol_skew_ratio
decade				
1970	187200.0	1377365.79	5.46	3.87
1980	64700.0	4510773.57	19.85	12.95
1990	62200.0	5834490.50	25.72	13.89
2000	133500.0	9231939.47	18.22	10.37
2010	165400.0	17708268.18	13.32	7.98

Decade comparison

The dataset was segmented on `(index.year // 10) * 10`.

Coverage

```
: #Rebuild the segments after cleaned
decades = {d: g for d, g in stock_prices.groupby('decade')}
for d, df in decades.items():
    print(f"{d}s - {df.shape[0]:,} rows, {df['ticker'].nunique()} tickers")
```

```
1970s - 164,499 rows, 93 tickers
1980s - 1,470,571 rows, 901 tickers
1990s - 3,427,832 rows, 2117 tickers
2000s - 6,572,053 rows, 3429 tickers
2010s - 9,338,934 rows, 5685 tickers
```

```
#Check by decades
for d, df in decades.items():
    print(f"{d}s - mean close: {df['close'].mean():.2f}, tickers: {df['ticker'].nunique()}")
```

```
1970s - mean close: 12.49, tickers: 93
1980s - mean close: 16.82, tickers: 901
1990s - mean close: 45.30, tickers: 2117
2000s - mean close: 122.44, tickers: 3429
2010s - mean close: 65.28, tickers: 5685
```

Observations rise 57-fold and the ticker count 61-fold. Each decade therefore describes a **different population of securities**, so all comparisons below are between samples, not a fixed panel tracked over time.

Closing price

```
summary = stock_prices.groupby('decade')[cols].agg(['mean', 'median', 'std']).round(2)
print(summary)
```

	open			high			low	
	mean	median	std	mean	median	std	mean	median
decade								
1970	12.49	2.79	46.98	12.64	2.84	47.48	12.37	2.77
1980	16.81	5.00	573.05	17.10	5.08	582.81	16.56	4.96
1990	45.29	10.58	1391.11	46.87	10.75	1464.63	43.81	10.44
2000	122.30	16.58	4485.64	125.65	16.87	4703.97	119.17	16.30
2010	65.26	20.01	1820.18	66.72	20.28	1941.57	63.92	19.75

		close			volume		
	std	mean	median	std	mean	median	std
decade							
1970	46.51	12.49	2.79	46.99	724893.64	187200.0	1377365.79
1980	562.26	16.82	5.00	575.03	837523.35	64600.0	4510088.44
1990	1329.33	45.30	10.59	1391.58	863527.19	62200.0	5834011.90
2000	4318.82	122.44	16.58	4509.78	1383562.38	133400.0	9230142.84
2010	1763.14	65.28	20.01	1850.13	1320505.22	165400.0	17707888.21

Name Ayleen Santander

GitHub Link:

https://github.com/AyleenSantander/M.11_Intro_AI/blob/main/Project.1_Data_Collection_and_Initial_Analysis_of_Stock_Market.pdf

The **median rises monotonically** = 2.79 to 20.01, a 7.2-fold increase over five decades. This is the trustworthy figure.

The **mean does not**. It peaks at 122.44 in the 2000s and falls to 65.28 in the 2010s, moving opposite to the median. A mean diverging from the median that sharply signals extreme values, not a genuine reversal in prices.

Trading volume

```
#Compare the summary statistics across decades and document any notable trends or changes in stock price
#behaviors and trading volumes.
cols = ['open', 'high', 'low', 'adj_close', 'volume']

comparison = stock_prices.groupby('decade').agg(
    n_rows=('adj_close', 'size'),
    n_tickers=('ticker', 'nunique'),
    close_mean=('adj_close', 'mean'),
    close_median=('adj_close', 'median'),
    close_std=('adj_close', 'std'),
    vol_mean=('volume', 'mean'),
    vol_median=('volume', 'median'),
    vol_std=('volume', 'std')
).round(2)

comparison['close_cv'] = (comparison['close_std'] / comparison['close_mean']).round(2)
comparison['vol_skew_ratio'] = (comparison['vol_mean'] / comparison['vol_median']).round(2)

print(comparison)
```

	n_rows	n_tickers	close_mean	close_median	close_std \
decade					
1970	164499	93	7.310000e+00	0.35	3.993000e+01
1980	1470571	901	2.112523e+15	1.18	1.727534e+17
1990	3427832	2117	1.476753e+09	4.87	7.314168e+11
2000	6572053	3429	6.844700e+02	11.66	5.075891e+04
2010	9338934	5685	6.246000e+01	17.79	1.841540e+03

	vol_mean	vol_median	vol_std	close_cv	vol_skew_ratio
decade					
1970	724893.64	187200.0	1377365.79	5.46	3.87
1980	837523.35	64600.0	4510088.44	81.78	12.96
1990	863527.19	62200.0	5834011.90	495.29	13.88
2000	1383562.38	133400.0	9230142.84	74.16	10.37
2010	1320505.22	165400.0	17707888.21	29.48	7.98

This is the most striking result in the project. **Median volume fell**, 187,200 in the 1970s down to 165,400 in the 2010s, while the standard deviation rose almost **thirteenfold**, from 1.38 million to 17.71 million.

The typical security does not trade more heavily today than in 1970. What changed is the gap between the busiest and quietest names.

The mean-to-median ratio traces the mechanism: 3.87 in the 1970s reflects a sample of 93 large, liquid, similar firms. It jumps past 13 by the 1990s as thousands of thinly traded small-caps enter alongside a handful of dominant mega-caps. The subsequent decline to 7.98 suggests liquidity has broadened across more names since 2000 rather than concentrating further.

Adjusted close, after cleaning

Name Ayleen Santander

GitHub Link:

https://github.com/AyleenSantander/M.11_Intro_AI/blob/main/Project.1_Data_Collection_and_Initial_Analysis_of_Stock_Market.pdf

```
#New Summary from the cleaned frame
comparison = clean_prices.groupby('decade').agg(
    n_rows=('adj_close', 'size'),
    n_tickers=('ticker', 'nunique'),
    close_mean=('adj_close', 'mean'),
    close_median=('adj_close', 'median'),
    close_std=('adj_close', 'std'),
    vol_mean=('volume', 'mean'),
    vol_median=('volume', 'median'),
    vol_std=('volume', 'std')
).round(2)

comparison['close_cv'] = (comparison['close_std'] / comparison['close_mean']).round(2)
comparison['vol_skew_ratio'] = (comparison['vol_mean'] / comparison['vol_median']).round(2)

print(comparison)
```

	n_rows	n_tickers	close_mean	close_median	close_std	vol_mean \
decade						
1970	164499	93	7.31	0.35	39.93	724893.64
1980	1470110	900	9.22	1.18	183.06	837750.58
1990	3427258	2117	30.29	4.87	779.01	863665.52
2000	6569431	3429	58.05	11.65	1057.91	1384100.65
2010	9338531	5685	52.54	17.79	699.74	1320560.15

	vol_median	vol_std	close_cv	vol_skew_ratio
decade				
1970	187200.0	1377365.79	5.46	3.87
1980	64700.0	4510773.57	19.85	12.95
1990	62200.0	5834490.50	25.72	13.89
2000	133500.0	9231939.47	18.22	10.37
2010	165400.0	17708268.18	13.32	7.98

Before cleaning, the 1980s mean read 2.11×10^{15} and the 1990s CV reached 495. After removing 4,059 rows the figures become interpretable, which is itself the strongest argument for having investigated rather than ignored them.

The coefficient of variation peaks at 25.72 in the 1990s then falls steadily to 13.32, roughly half. Relative to the mean, the spread between cheap and expensive securities has narrowed.

Cross-cutting pattern: price dispersion and volume dispersion move in opposite directions after 2000. Price CV falls from 18.22 to 13.32 while volume standard deviation nearly doubles from 9.2 million to 17.7 million. Securities have become more alike in price and less alike in how actively they trade.

Part 14: Summary of EDA findings

Data structure

Two datasets were analysed. `historical_stocks.csv` holds 6,460 securities across 5 descriptive columns. `historical_stock_prices.csv` holds 20,973,889 daily observations across 8 columns, occupying 1.3 GB in memory. They join one-to-many on `ticker`, and because no ticker repeats in the reference table, the join produces no row inflation. Coverage runs from 2 January 1970 to 24 August 2018.

Data quality

Missing values. None in the price table. 1,440 securities (22.3%) lack a sector, which propagates to 2,549,449 price rows (12.2%) after the join. These are predominantly funds and trusts with no operating sector, so the blanks are correct rather than defective. All were retained.

Duplicates. None found, on either the full row or the `ticker + date` business key. The file is well formed at the record level.

Data-quality issues. A `DtypeWarning` on load flagged `adj_close` as mixed-type. Investigation traced this to impossible adjusted closing prices reaching 1.89×10^{19} , concentrated in just ten securities, with YRCW, ELJ and AAN alone accounting for 76% of affected rows. On those same rows `close` is entirely valid, which localises the fault to the adjustment factor rather than the underlying price data.

Cleaning decisions. 4,059 rows (0.019%) were excluded from `adj_close` analysis. No rows were deleted from the dataset itself, and the extreme `close` values in Part 9 were retained because reverse-split back-adjustment remains a plausible explanation that the data alone cannot rule out.

Numerical analysis

Every numerical variable is heavily right-skewed. The mean closing price of 76.11 sits 4.9 times above the median of 15.45. Volume is worse: a mean of 1,227,043 against a median of 126,000. Standard deviations exceed the means in every case.

The practical consequence, carried through the whole project: **the median is the appropriate summary statistic for this dataset**. Where the mean and median disagree, the median is the one that describes a typical security.

Outliers and anomalies

`close` reaches 1,779,750 (SCON, February 2000) on 400 shares traded. Four securities, SCON, CODA, TVIX and TOPS, account for the entire top of the distribution, all on volumes between 34 and 400 shares. The internal price relationships hold in every row, so these are coherently scaled values rather than random corruption, most likely reverse-split artefacts.

`adj_close` contains values that are impossible under any interpretation. Ten securities are affected, in contiguous date blocks, with 2,621 of 4,059 bad rows falling in the 2000s.

The most consequential finding for method: `adj_close`, normally the *preferred* column for historical analysis because it accounts for splits and dividends, is **less reliable than raw `close` in this dataset**. That inverts the standard assumption and should be stated in any downstream work.

Visual analysis

The decade time-series charts required two corrections before they showed anything real. Plotted as unweighted means on uncleaned data, they produced impossible spikes, 460.32 in February 2000 against neighbouring months near 65, and buried the actual market history. Rebuilt as medians on cleaned data, four genuine drawdowns became visible where the mean had shown two:

Event	Peak	Trough	Decline
1973–74	5.08 (Jan 1973)	1.88 (Oct 1974)	–63%
1987	6.72 (Aug 1987)	4.72 (Nov 1987)	–30%
2000–02	14.88 (Apr 1998)	11.42 (Jan 2003)	–23%
2007–09	23.93 (Nov 2007)	10.02 (Feb 2009)	–58%

The 1987 crash was **completely absent** from the mean series and appeared only after switching to the median. The dot-com bust barely registers for the typical security at –23%, because the bubble was concentrated in a narrow set of technology names the median company never participated in.

A log-scaled median series across the full 1970–2018 span shows steady proportional growth rather than acceleration, and shows that the typical security took roughly seven years, until about 2014, to regain its 2007 level.

Histograms confirmed the skew visually: `close` produces one bar and a scatter of single observations out past 400,000 dollars, readable only because the count axis is logarithmic. `volume`, plotted as $\log_{10}$, is near-symmetric and bell-shaped in every decade, trading volume is approximately log-normal, skewed in raw units and well behaved once transformed.

Boxplots showed the same contrast. The `close` boxplot collapses to a flat line with a dense column of outliers above it and conveys almost nothing. The log-volume boxplot by decade works well and reproduces the summary-statistics finding directly: medians dip in the 1980s and 1990s then recover, while the boxes widen steadily from the 1970s onward.

The single-stock series (KO) showed a flat 1970–1985 period under 2 dollars, sustained growth to about 43 by 1998, a decline to roughly 20 by 2003, and recovery to about 46 by 2018. Neither the 1973–74 bear market nor the 1987 crash is visible, because at a split-adjusted price under 2 dollars those moves are imperceptible on a linear axis reaching 50, a reminder that axis choice determines what a chart can show.

Initial hypotheses

Six hypotheses follow from the findings above, each with a proposed test. None is established by this EDA.

H1 = Market structure, not company behaviour, drives the widening volume dispersion.

Median volume fell slightly while its standard deviation rose thirteenfold. If this reflects the arrival of thousands of thinly traded small-caps rather than a change in how existing firms trade, then restricting to securities present in all five decades should show a much flatter dispersion trend. *Test:* build a fixed panel of securities appearing in every decade and recompute the volume statistics.

H2 = Sector composition explains part of the apparent price growth. The rising median could reflect a shift in which kinds of companies are listed rather than price appreciation within sectors. *Test:* compute the median close by sector by decade using the merged table, and check whether the within-sector trend matches the overall one.

H3 = Price level predicts volatility. Low-priced securities plausibly show larger percentage swings than high-priced ones. *Test:* bucket securities by median price and compare the distribution of daily returns across buckets.

H4 = Volume spikes precede large price moves. If unusual volume leads rather than follows price changes, it has predictive content. *Test:* compute the correlation between volume on day t and absolute return on day $t+1$, per security, and test whether the mean correlation differs from zero.

H5 = The intraday range (high – low) widens during market-wide drawdowns. The four drawdowns identified in the median series give natural test windows. *Test:* compute $(\text{high} - \text{low}) / \text{close}$ by month and compare drawdown periods against adjacent calm periods.

H6 = Securities with corrupt adjustment factors share a corporate-action characteristic. All ten affected tickers may have undergone reverse splits or similar events that the data provider's adjustment pipeline mishandled. *Test:* check the split history of YRCW, ELJ, AAN, MBVX, SAB, TVIX, CODA, CTIC, ABIO and SCON against a second source. This one also has practical value, if confirmed, it gives a rule for detecting the same fault in other datasets.

H1 and H6 are the most valuable next steps. H1 because every decade comparison in this report is confounded by changing composition until it is answered; H6 because it would convert a threshold filter into a principled correction.

Limitations

Survivorship bias. The reference table contains only securities listed as of 2018. Companies that went bankrupt, were acquired or were delisted are absent from it, and their price histories are

correspondingly under-represented. Every conclusion about long-run price levels is therefore biased upward. This is the single most important constraint on the analysis.

Changing composition. The ticker universe grows 61-fold across the five decades. No decade comparison holds the sample constant, so market evolution and sampling change are confounded throughout. Restricting to securities present in all five decades would resolve this at the cost of a much smaller sample.

Nominal dollars. No inflation adjustment was applied. A 1970 dollar is worth roughly six to seven 2018 dollars, so a substantial part of the observed price growth is currency debasement rather than real appreciation.

Removed observations. 4,059 rows were excluded via a threshold filter on `adj_close`. The cut-off of 100,000 is defensible but arbitrary, and a different threshold would give slightly different figures. The exclusion is documented, not hidden.

Retained anomalies. The extreme `close` values in Part 9 were kept because the data alone cannot distinguish a reverse-split artefact from an error. They continue to influence every mean-based statistic in the report, which is part of why medians are preferred throughout.

Unverified corporate actions. No split or dividend history was available, so `adj_close` could not be validated against `close` on the rows where they disagree.

Sector coverage. 12.2% of price observations carry no sector label, so any sector-level aggregation silently excludes them.